Dataset / Notebook Export

Content extracted from the uploaded Jupyter Notebook. Original cell order is preserved; code cells and written responses are included.

Cell 1 — Markdown

```
1. Dynamic Programming for Matrix Chain Multiplication (25 poin)
Misalkan A1, A2, A3, A4, A5 adalah matriks berukuran 4 × 7, 7 × 5, 5 × 10, 10 × 3, dan 3 × 9 berturut-turut

(a) (3 poin) Asumsikan untuk menghitung hasil perkalian matriks XY dengan X berukuran m × n dan Y
    berukuran n×k dibutuhkan m·n·k operasi perkalian aritmetika. Berapa operasi perkalian aritmetikakah
    yang dilakukan untuk menghitung A1A2A3A4A5 jika urutan perkaliannya adalah sesuai dengan prioritas
    berdasarkan tanda kurung: ((A1(A2A3))(A4A5))?

kita lakukan perkalian step by step.
    1. (A2A3) = 7 x 5 x 10
    2. A1(1) = 4 x 7 x 10
    3. (A4A5) = 10 x 3 x 9
    4. (2)(3) = 4 x 10 x 9

Totalnya adalah = 350 + 280 + 270 + 360
                  = 1260
```

Cell 2 — Markdown

```
(b) (7 poin) Simulasikan algoritma Dynamic Programming pada salindia daa9 untuk mencari banyak perkalian
    aritmetika minimum yang dilakukan untuk mengalikan A1A2A3A4A5. Buat tabelnya dan berikan penjelasan urutan pengisiann

Berdasarkan video yang saya tonton dari youtube : https://www.youtube.com/watch?v=prx1psByp7U
    Saya membuat tabel seperti dibawah ini :
[Image:  — https://imgur.com/j522xAD.png]
```

Dengan perhitungan :

```
Dilakukan perhitungan dengan rumus M[i,j] = min { M[i,k], M[k+i,j] + di-1.dk.dj }
    Dengan klasifikasi :
    d0 = 4
    d1 = 7
    d2 = 5
    d3 = 10
    d4 = 3
    d5 = 9
```

Lalu pertama tama kita isi terlebih dahulu M[1,1], M[2,2], M[3,3], M[4,4] dan M[5,5] dengan 0

Selanjutnya kita lakukan perhitungan satu persatu

```
M[1,2] = 4 x 7 x 5
           = 140
    M[2,3] = 7 x 5 x 10
           = 350
    M[3,4] = 5 x 10 x 3
           = 150
    M[4,5] = 10 x 3 x 9
           = 270
    M[1,3] = min {
             M[1,1] + M[2,3] + 4x7x10 = 0 + 350 + 280 = 630,
             M[1,2] + M[3,3] + 4x5x10 = 140 + 0 + 200 = 340 }
           = min { 630, 340 }
           = 340
    M[2,4] = min {
             M[2,2] + M[3,4] + 7x5x3 = 0 + 150 + 105 = 255,
             M[2,3] + M[4,4] + 7x10x3 = 350 + 0 + 210 = 560}
           = min { 255, 560 }
           = 255
    M[3,5] = min {
             M[3,3] + M[4,5] + 5x10x9 = 0 + 270 + 450 = 720,
             M[3,4] + M[5,5] + 5x3x9 =150 + 0 + 135 = 285}
           = min { 720, 285 }
           = 285
    M[1,4] = min {
             M[1,1] + M[2,4] + 4x7x3 = 0 + 255 + 84 = 339,
             M[1,2] + M[3,4] + 4x5x3 = 140 +150 + 60 = 350,
```

```
              M[1,3] + M[4,4] + 4x10x3 = 340 + 0 + 120 = 460 }
           = min { 339, 350, 460 }
           = 339
    M[2,5] = min {
              M[2,2] + M[3,5] + 7x5x9 = 0 + 285 + 315 = 600,
              M[2,3] + M[4,5] + 7x10x9 = 350 + 270 + 630 = 1250,
              M[2,4] + M[5,5] + 7x3x9 = 255 + 0 + 189 = 444 }
           = min { 600, 1250, 444 }
           = 444
    M[1,5] = min {
              M[1,1] + M[2,5] + 4x7x9 = 0 + 444 + 252 = 696,
              M[1,2] + M[3,5] + 4x5x9 = 140 + 285 + 180 = 605,
              M[1,3] + M[4,5] + 4x10x9 = 340 + 270 + 360 = 970,
              M[1,4] + M[5,5] + 4x3x9 = 339 + 0 + 108 = 447}
           = min { 696, 605, 970, 447 }
           = 447
```

Cell 3 — Markdown

```
(c) (5 poin) Berikan tanda kurung pada ekspresi A1A2A3A4A5 untuk mengindikasikan urutan perkalian
    yang minimum banyak perkalian aritmetiknya berdasarkan soal (b). Jelaskan bagaimana cara Anda
    mendapatkannya.
```

Untuk A1A2A3A4A5 ada 5 kemungkinan yaitu :

```
1. ((A1(A2(A3A4)))A5) = (5 x 10 x 3) + (7 x 5 x 3) + (4 x 7 x 3) + (4 x 3 x 9)
                      = 150 + 105 + 84 + 108
                      = 447
    2. (((A1(A2A3))A4)A5) = (7 x 5 x 10) + (4 x 7 x 10) + (4 x 10 x 3) + (4 x 3 x 9)
                      = 350 + 280 + 120 + 108
                      = 858
    3. ((A1((A2A3)A4))A5) = (7 x 5 x 10) + (7 x 10 x 3) + (4 x 7 x 3) + (4 x 3 x 9)
                      = 350 + 210 + 84 + 108
                      = 752
    4. (((A1A2)(A3A4))A5) = (4 x 7 x 5) + (5 x 10 x 3) + (4 x 5 x 3) + (4 x 3 x 9)
                      = 140 + 150 + 60 + 108
                      = 458
    5. ((((A1A2)A3)A4)A5) = (4 x 7 x 5) + (4 x 5 x 10) + (4 x 10 x 3) + (4 x 3 x 9)
                      = 140 + 200 + 120 + 108
                      = 568
    Sehingga dapat dilihat bahwa urutan perkalian yang memiliki minimum banyak perkalian adalah ((A1(A2(A3A4)))A5) sebany

Hal di atas juga dapat dibuktikan dengan cara yang kita lakukan di nomor 1b yaitu dengan :
    1. ((A1(A2(A3A4)))A5) = M[1,4] + M[5,5] + 4x3x9 = 339 + 0 + 108 = 447
    2. ((A1(A2A3))(A4A5)) = M[1,3] + A[4,5] + 4x10x9 = 340 + 270 + 360 = 970
    3. (A1(A2((A3A4)A5))) = M[1,1] + M[2,5] + 4x7x9 = 0 + 444 + 252 = 696
```

Dengan 3 contoh diatas saja sudah cukup membuktikan bahwa minimum banyak perkalian adalah 447. Sebenarnya kita dapat menyimpulkannya karena kita sudah mengetahui bahwa kombinasi terkecil adalah M[1,4] + M[5,5]. Misal kita cari kembali kombinasi terkecil M[1,4] dan ditemukan adalah M[1,1] + M[2,4]. Setelah itu kita cari kembali kombinasi terkecil M[2,4] dan ditemukan M[2,2] + M[3,4] dan juga didapatkan ((A1(A2(A3A4)))A5).

Note : kombinasi terkecil bisa dilihat dari tabel yang sudah kita buat di 1b yaaa

Cell 4 — Markdown

```
(d) (10 poin) Asumsikan bahwa terdapat suatu algoritma yang dapat mengalikan dua matriks XY dengan
    X berukuran m × n dan Y berukuran n × k hanya dengan mnk − mk operasi perkalian aritmetika,
    alih-alih mnk. Apakah urutan perkalian yang paling minimum jumlah operasi perkalian aritmetikanya
    berubah? Gunakan dynamic programming lagi untuk membantu Anda mendapatkan jawabannya.

Kita dapat melakukannya dengan hal yang sama dengan 1b dan 1c. Pertama tama kita buat terlebih dahulu tabel seperti 1b da
[Image:  — https://imgur.com/82NDP7L.jpg]

Dengan perhitungan satu persatu :
    M[1,2] = 4x7x5 – 4x5
           = 120
    M[2,3] = 7x5x10 – 7x10
           = 280
    M[3,4] = 5x10x3 – 5x3
           = 135
    M[4,5] = 10x3x9 – 10x9 = 180
           = 270
    M[1,3] = min {
              M[1,1] + M[2,3] + 4x7x10 – 4x10 = 0 + 280 + 240 = 520
              M[1,2] + M[3,3] + 4x5x10 – 4x10 = 120 + 0 + 160 = 280  }
           = min { 520, 280 }
```

```
          = 280
    M[2,4] = min {
              M[2,2] + M[3,4] + 7x5x3 – 7x3 = 0 + 135 + 84 = 219
              M[2,3] + M[4,4] + 7x10x3 – 7x3 = 280 + 0 + 189 = 469}
          = min { 219, 469 }
          = 219
    M[3,5] = min {
             M[3,3] + M[4,5] + 5x10x9 – 5x9 = 0 + 180 + 405 = 585
             M[3,4] + M[5,5] + 5x3x9 – 5x9 = 135 + 0 + 90 = 225}
          = min { 585, 225 }
          = 225
    M[1,4] = min {
              M[1,1] + M[2,4] + 4x7x3 – 4x3 = 0 + 219 + 72 = 291
              M[1,2] + M[3,4] + 4x5x3 – 4x3 = 120 + 135 + 48 = 303
              M[1,3] + M[4,4] + 4x10x3 – 4x3 = 280 + 0 + 108 = 388  }
          = min { 291, 303, 388 }
          = 291
    M[2,5] = min {
              M[2,2] + M[3,5] + 7x5x9 – 7x9 = 0 + 225 + 252 = 477
              M[2,3] + M[4,5] + 7x10x9 – 7x9 = 280 + 180 + 567 = 1027
              M[2,4] + M[5,5] + 7x3x9 – 7x9 = 219 + 0 + 126 = 345 }
          = min { 477, 1027, 345 }
          = 345
    M[1,5] = min {
              M[1,1] + M[2,5] + 4x7x9 – 4x9 = 0 + 345 + 216 = 561
              M[1,2] + M[3,5] + 4x5x9 – 4x9 = 120 + 225 + 144 = 489
              M[1,3] + M[4,5] + 4x10x9 – 4x9 = 280 + 180 + 324 = 784
              M[1,4] + M[5,5] + 4x3x9 – 4x9 = 291 + 0 + 72 = 363}
          = min { 561, 489, 784, 363 }
          = 363

Setelah mendapatkan nilai dalam tabel, kita dapat melakukan hal pada part 1c yaitu :
    1. (A1((A2(A3A4)A5)) = M[1,1] + M[2,5] + (4x7x9)-(4x9)
                         = 0 + 345 + 252 - 36
                         = 561
    2. ((A1(A2A3))(A4A5)) = M[1,3] + M[4,5] + (4x10x9)-(4x9)
                          = 280 + 180 + 360 - 36
                          = 784
    3. ((A1(A2(A3A4)))A5) = M[1,4] + M[5,5] + (4x3x9) - (4x9)
                          = 291 + 0 + 108 - 36
                          = 363
    4. ((A1A2)((A3A4)A5)) = M[1,2] + M[3,5] + (4x5x9)-(4x9)
                          = 120 + 225 + 18 - 36
                          = 489
    Maka didapatkan nilai terkecilnya untuk M[1,5] adalah 363 dengan urutan ((A1(A2(A3A4)))A5)
```

Cell 5 — Markdown

```
2. Shortest Superstring Problem (25 poin)
Diberikan himpunan S yang terdiri dari n buah string sehingga tiap anggota himpunan S bukanlah substring
dari anggota lainnya. Shortest Superstring Problem mencari string terpendek s
yang mengandung tiap string
di S sebagai substring dari s
Diberikan algoritma SSS berikut yang menerima S sebagai input:
SSS(S):
1 T = S
2 while |T| > 1 do:
3 Cari a dan b, yaitu dua string yang paling banyak overlap-nya di T
4 Ganti a dan b dengan hasil melakukan overlap antara a dan b
5 return T

(a) (7 poin) Rancanglah suatu algoritma (dalam bentuk pseudocode) yang dapat digunakan pada line 3.
    Jelaskan cara kerja algoritma Anda
```

Cell 6 — Code

```
SSS(S):
    int max = min value untuk menyimpan max overlap;
    String str1, str2 = "";
    T = S;
    t = |T|
    while t < 1 {
            for i (1 sampai t){
        for j (i + 1 sampai t){
            if (i != j){
                a = T[i];
                b = T[j];
```

```
                for k (1 sampai nilai min dsri length a dan b){
                    suffix = a[length dari a dikurang dengan k]
                    prefix = b[0 sampai k]
                    if (k nilainya sama dengan nilai min dari length a dan b){
                        if (tidak ada persamaan dari suffix dan prefix){
                            max = 0;
                            str1 = T[i]
                            str2 = T[j]
                        }
                    else if (suffix dan prefix sama dan max < k){
                        max = k
                        str1 = T[i]
                        str2 = T[j]
                    }
                }
            }
        }
            else {
                break
            }
        }
    }

Sumber : https://www.geeksforgeeks.org/shortest-superstring-problem/
```

Cell 7 — Markdown

```
(b) (3 poin) Jalankan algoritma SSS(S) dengan input S = {xyy, yyy, xyz, yyz, yzz}
    Misalkan kita menjalankan algoritma SSS(S) dengan S = {xyy, yyy, xyz, yyz, yzz}
    SSS(xyy, yyy, xyz, yyz, yzz) maka akan kita dapatkan xyyyzzxyz
```

Sumber bacaan : https://www.geeksforgeeks.org/shortest-superstring-problem/

```
Output yang bisa didapatkan dengan step by step adalah :
    - ketika S = {xyy, yyy, xyz, yyz, yzz}, ketika kita membandingkan variabel variabel yang ada, kita akan menemukan bah
    - ketika S = {xyyy, xyz, yyz, yzz}, ketika kita membandingkan variabel variabel yang ada, kita akan menemukan bahwa x
    - ketika S = {xyyyz, xyz, yzz}, ketika kita membandingkan variabel variabel yang ada, kita akan menemukan bahwa xyyyz
    - Terakhir, ketika S = {xyyyzz,xyz}, ketika kita membandingkan variabel variabel yang ada, tidak ada persamaan dari p
```

Cell 8 — Markdown

```
(c) (5 poin) Analisis kompleksitas waktu terburuk dari algoritma pada poin (a).
    Menurut saya kompleksitas waktu terburuknya adalah n^2k. Hal ini saya dapatkan dari membaca sumber geeks for geeks da
```

Cell 9 — Markdown

```
(d) (3 poin) Berdasarkan jawaban pada poin (c), berikan analisis kompleksitas waktu terburuk dari algoritma SSS(S).
    Dengan tambahan while loop pada soal nomor 2, while ini digunakan untuk mengurangi ukuran dari array untuk setiap ite
```

Cell 10 — Markdown

(e) (2 poin) Paradigma apa yang digunakan oleh algoritma SSS(S)? Jelaskan.

Paradigma yang digunakan adalah Greedy paradigma. Sesuai yang mungkin juga sudah dijelaskan di kelas, algoritma ini menggunakan greedy paradigma karena setiap kita melakukan merge terhadap dua string dengan panjang maksimal yang overlap, hapus dari array string dan kita letakkan merged string ke array string.

Sumber : https://www.geeksforgeeks.org/shortest-superstring-problem/

Cell 11 — Markdown

```
(f) (5 poin) Apakah solusi yang dihasilkan algoritma SSS(S) optimal? Jika ya, berikan pembuktiannya.
    Jika tidak, berikan sebuah contoh penyangkal.

Dengan algoritma SSS(S), ada kemungkinan bahwa dapat juga tidak optimal. Misalnya kita punya input S = {abb, bbc, bab} (s
```

Padahal, kita bisa saja menggabungkan terlebih dahulu antara abb dan bab sehingga akan didapatkan abab, lalu ketika kita cek kembali S = {abab, bbc}, ada persamaan dan akan mengupdate saver menjadi ababc. Hal ini tidak akan didapatkan dengan greedy paradigma.

Cell 12 — Markdown

3. Gajah, Kuda, dan Benteng (30 poin)
Baris-baris papan catur berukuran 5×5 dilabeli 1, 2, . . . , 5 dari atas ke bawah dan kolom-kolomnya 1, 2, . . . , 5 dari kiri ke kanan; setiap kotak di baris i dan kolom j diberi label (i, j).
Gajah adalah bidak catur yang jika berada pada kotak (i, j) dapat menyerang semua kotak (k, l) dengan |i − k| = |j − l|.
Sementara itu, kuda adalah bidak catur yang jika berada pada kotak (i, j) dapat menyerang semua kotak (k, l) dengan
{|i − k|, |j − l|} = {1, 2}.
1
Benteng adalah bidak catur yang jika berada pada kotak (i, j) dapat menyerang semua kotak (k, l) dengan i = k atau j = l.
Anda diminta menentukan banyaknya cara meletakkan pada papan catur 3 bidak berikut sehingga tidak ada bidak yang menyerang bidak lain:
Digit terakhir NPM 3: 0 kuda, 1 benteng, 2 gajah.
Anda diminta mengimplementasikan algoritma backtracking dalam sebuah program berbahasa Python untuk menjawab pertanyaan di atas. Jawab pula pertanyaan berikut untuk membantu Anda merancang algoritma tersebut:

(a) (4 poin) Bagaimana cara Anda merepresentasikan solusinya? Jelaskan menggunakan representasi/struktur data apa saja ya
matriks

Cell 13 — Markdown

(b) (4 poin) Apa saja parameter dari algoritma rekursif Anda? Berikan penjelasan apa yang disimpan di masing-masing parameter di setiap pemanggilan rekursif. Bagaimana pemanggilan yang dilakukan untuk menyelesaikan masalah di atas?

Cell 14 — Markdown

(c) (10 poin) Jelaskan implementasi Anda dan berapakah jawaban akhirnya?

Cell 15 — Markdown

(d) (5 poin) Berikan ilustrasi dari eksekusi program Anda menggunakan pohon rekursi hingga ditemukan setidaknya dua buah solusi. Jika tidak ada solusi, jelaskan mengapa tidak ada solusi

Cell 16 — Markdown

(e) (7 poin) Apakah algoritma backtracking Anda dapat dibuat lebih efisien dynamic programming atau memoization? Jika bisa, jelaskan bagaimana dan jelaskan seberapa efisien jadinya? Jika tidak bisa, jelaskan mengapa

Cell 17 — Markdown

4. Seorang kurir dari suatu online shop diminta untuk mengantarkan beberapa paket ke beberapa rumah. Agar hemat bensin, dia ingin merencanakan agar rute pengantaran paket-paket tersebut sependek mungkin. Rute pengantarannya dimulai dari dan berakhir di kantor online shop. Kurir tersebut juga hanya mau mengunjungi setiap rumah tepat satu kali saja, dan tidak boleh ada rumah yang tidak ia kunjungi.
Jika diketahui daftar n−1 rumah yang harus dikunjungi (tidak termasuk kantor online shop) {a1, a2, . . . , an−1}, serta jarak antara setiap rumah dengan rumah lainnya dan jarak antara kantor online shop (a0) dan masingmasing rumah {d(a , aj)} dengan i, j = 0, . . . , n − 1 dan i < j, bantulah kurir tersebut merencanakan rute perjalanannya dengan menjawab soal-soal berikut:

(a) (3 poin) Masalah di atas merupakan contoh dari salah satu permasalahan abstrak yang well-known di bidang ilmu teori komputasi. Sebutkan nama masalahnya dan spesifikasi deskripsi masalahnya secara abstrak (apa masukan dan keluarannya). Jelaskan bagaimana masalah di atas merupakan contoh dari permasalahan abstrak ini. (Abstrak di sini maksudnya umum; misalnya menggunakan bahasa teori graf.)

Sumber : https://www.geeksforgeeks.org/travelling-salesman-problem-set-1/
https://www.geeksforgeeks.org/traveling-salesman-problem-tsp-implementation/

Nama dari problem abstrak di atas adalah The Travelling Salesman Problem (TSP)
The Travelling Salesman Problem menjelaskan mengenai sekumpulan kota dan jarak antara setiap pasangan kota dengan mas

Cell 18 — Markdown

```
(b) (2 poin) Ada berapa banyak kemungkinan rute yang bisa diambil kurir tersebut, jika rute yang diambil
    tidak harus yang terpendek? Tulis secara eksak, jangan menggunakan notasi asimtotik.

Sumber : https://math.libretexts.org/Bookshelves/Applied_Mathematics/Math_in_Society_(Lippman)/06%3A_Graph_Theory/6.06%3A
    Setelah kita menemukan bahwa problem diatas merupakan TSP, kita dapat melihat kembali sumber yang tertera bahwa ada d
    - Untuk simpul dalam graf lengkap, akan ada (n-1)! = (n-1)(n-2)(n-3)... 3.2.1 routes. Hal ini dikhususkan jika semisa
    - Jika reverse order tidak perlu dibaca, maka kita dapat menyimpulkan bahwa terdapat (n-1)!/2 unique route
```

Cell 19 — Markdown

```
(c) (2+3+3+10 poin) Permasalahan ini dapat diselesaikan dengan pendekatan branch and bound.
    i. Apa yang menjadi node dan edge pada tree yang mengilustrasikan solusi dari permasalahan ini?
    Sumber : https://www.geeksforgeeks.org/traveling-salesman-problem-using-branch-and-bound-2/

Ketika kita melakukan pendekatan branch and bound, kita selalu mengetahui bahwa pada tree, root node nya merupakan dimana
    Pada sumber di atas, tertera contoh kemungkinan yang terjadi yaitu node 0 - node 1 - node 2 - node 3 - node 0

ii. Jelaskan cara menghitung atau menentukan lower bound pada suatu node di level 1 dan level 2, apabila root node adalah
    Sumber :https://www.geeksforgeeks.org/traveling-salesman-problem-using-branch-and-bound-2/

Level 1 akan enumerate seluruh vertices yang mungkin kita jangkau seperti node 1, node 2, node 3, ... node n - 1 untuk gr
```

Pada level 2, kita lagi lagi enumerate seluruh vertices yang mungkin kita jangkau. Untuk kasus diatas setelah node 1, kita akan mengecek node 2, node 3, ... node n-1. Pertimbangkan lower bound untuk node 2 saat kita pindahkan dari node 0 ke node 1, kita include edge node 1 kd node 2 ke rute dan ubah lower bound dari node ini yang dapat dihitung dengan rumus old lower bound - ((minimum cost edge kedua dari node 1 + minimum cost edge dari node 2)/2 + cost edge dari node 1 ke node 2

Sekian perhitungan lower bound dari level 1 dan level 2, perubahan yang ada hanya ada pada saat kita memasukkan minimum cost edge kedua dari node 1 karena minimum cost edge dari node 1 sudah kita kurangi pada level sebelumnya.

```
iii. Jelaskan bagaimana Anda memutuskan untuk melakukan pruning menggunakan lower bound yang
    Anda tetapkan tersebut.
    Sumber : https://www.geeksforgeeks.org/traveling-salesman-problem-using-branch-and-bound-2/
             http://www.enseignement.polytechnique.fr/informatique/INF431/X09-2010-2011/AmphiLL/branch_and_bound_for_TSP-n
```

Kita melakukan pruning dengan mneggunakan lower bound ketika lower bound yang kita dapatkan nilainya lebih besar daripada minimum cost dari rute terpendek yang kita miliki sekarang (current shortest route)

```
iv. Anda diminta mengimplementasikan algoritma dengan pendekatan tersebut dalam sebuah program
    berbahasa Python dan berikan penjelasan algoritma Anda. Luaran yang diharapkan adalah daftar
    urutan tempat yang dikunjungi (mulai dari a0 dan juga berakhir di a0), serta total jarak yang
    ditempuh kurir.
```

Sumber : https://www.geeksforgeeks.org/traveling-salesman-problem-using-branch-and-bound-2/

```
Saya mencoba untuk mengikuti angka pada video https://www.youtube.com/watch?v=vTfFhBDWbEE yaitu [[0, 132, 217, 164, 58],
       [132, 0, 290, 201, 79], [217, 290, 0, 113, 303], [164, 201, 113, 0, 196], [58, 79, 303, 196, 0]] dan hasilnya suda
    Penjelasan algoritma :
    - Terdapat fungsi copytofinal yang berfungsi untuk menyalin temporary solution ke final solution
    - Terdapat fungsi firstmin dengan dua variabel yaitu adj dan i yang berfungsi untuk mencari minimum edge cost dengan
    - Terdapat fungsi second min yang berfungsi untuk mencari secon minimum edge cost yamg memiliki end pada vertex i. Di
    - Terdapat fungsi TSPRec yang mengambil ergumen :
        curr_bound = lower bound dari node root
        curr_weight = menaruh weight dari path sejauh ini
        level = level saat ini ketika kita bergerak di search space tree
        curr_path[] = tempat solusinya disimpan dan nantinya akan di copy ke final path
        Kita memiliki base case yaitu ketika kita sampai ke level N, kita akan mengecek apakah ada edge dari vertex sebel
     - Didalamnya terdapat kondisi ketika vertex selanjutnya dipertimbangkan tidak sama (entri diagonal dalam matix adjac
     - ketika levelnya 1, kita menhitung perhitungan yang berbeda dari curr_bound untuk level 2 dari level lainnya
     - Untuk kondisi kainnya adalah ketika curr_bound ditambah curr_weight merupakan lower bound nya dan jika lower bound
     - Lalu kita lakukan reset terhadap array yang sudah kita visit
     - Lalu ada fungsi TSP (adj) yang melakukan set up final path. Didalamnya kita mengkalkulasi initial lower bound untu
     - Untuk urutannya kita melakukan inisiasi adj dengan N yang sesuai. Lalu panggil TSP(adj) dan print final res
```

Cell 20 — Code

```
import math
maxsize = float('inf')

def copyToFinal(curr_path):
    final_path[:N + 1] = curr_path[:]
    final_path[N] = curr_path[0]

def firstMin(adj, i):
    min = maxsize
```

```
    for k in range(N):
        if adj[i][k] < min and i != k:
            min = adj[i][k]

    return min

def secondMin(adj, i):
    first, second = maxsize, maxsize
    for j in range(N):
        if i == j:
            continue
        if adj[i][j] <= first:
            second = first
            first = adj[i][j]

        elif(adj[i][j] <= second and
             adj[i][j] != first):
            second = adj[i][j]

    return second

def TSPRec(adj, curr_bound, curr_weight,
           level, curr_path, visited):
    global final_res

    if level == N:

        if adj[curr_path[level - 1]][curr_path[0]] != 0:

            curr_res = curr_weight + adj[curr_path[level - 1]]\
                                        [curr_path[0]]
            if curr_res < final_res:
                copyToFinal(curr_path)
                final_res = curr_res
        return

    for i in range(N):
        if (adj[curr_path[level-1]][i] != 0 and
                            visited[i] == False):
            temp = curr_bound
            curr_weight += adj[curr_path[level - 1]][i]

            if level == 1:
                curr_bound -= ((firstMin(adj, curr_path[level - 1]) +
                                firstMin(adj, i)) / 2)
            else:
                curr_bound -= ((secondMin(adj, curr_path[level - 1]) +
                                  firstMin(adj, i)) / 2)

            if curr_bound + curr_weight < final_res:
                curr_path[level] = i
                visited[i] = True

                TSPRec(adj, curr_bound, curr_weight,
                       level + 1, curr_path, visited)

            curr_weight -= adj[curr_path[level - 1]][i]
            curr_bound = temp

            visited = [False] * len(visited)
            for j in range(level):
                if curr_path[j] != -1:
                    visited[curr_path[j]] = True
def TSP(adj):
    curr_bound = 0
    curr_path = [-1] * (N + 1)
    visited = [False] * N

    for i in range(N):
        curr_bound += (firstMin(adj, i) +
                       secondMin(adj, i))

    curr_bound = math.ceil(curr_bound / 2)
    visited[0] = True
    curr_path[0] = 0
    TSPRec(adj, curr_bound, 0, 1, curr_path, visited)

adj = [[0, 132, 217, 164, 58],
       [132, 0, 290, 201, 79],
       [217, 290, 0, 113, 303],
       [164, 201, 113, 0, 196],
```

```
       [58, 79, 303, 196, 0]]
N = 5
final_path = [None] * (N + 1)
visited = [False] * N

final_res = maxsize

TSP(adj)

print("Minimum cost :", final_res)
print("Path Taken : ", end = ' ')
for i in range(N + 1):
    print(final_path[i], end = ' ')

# This code is contributed by ng24_7

# Sumber : https://www.geeksforgeeks.org/traveling-salesman-problem-using-branch-and-bound-2/
```

Cell 21 — Markdown

```
(d) (7 poin) Selain pendekatan brute force pada bagian (b) dan pendekatan branch and bound pada bagian
    (c), apakah ada pendekatan lain untuk menyelesaikan permasalahan ini? Jika ya, jelaskan idenya secara
    singkat (tidak perlu membuat programnya). Jika tidak, jelaskan mengapa.

Sumber :
     https://ejournal.bsi.ac.id › article › download › pdf
                          https://en.wikipedia.org/wiki/Nearest_neighbour_algorithm#:~:text=The%20nearest%20neighbour
    http://digitalfirst.bfwpub.com/math_applet/asset/3/TSP_NN.html
     https://blog.routific.com/travelling-salesman-problem

Ada! Kita bisa menggunakan Nearest Neighbour method.  Pada link yang saya cantumkan diatas, tertulis bahwa terdapat 3 con
```

Untuk step by step nya adalah :

```
1. Inisialisasi semua veritces sebagai belum dikunjungi.
    2. Pilih vertices sembarang, atur sebagai vertices u. Tandai u telah dikunjungi.
    3. Temukan edge terpendek yang menghubungkan vertices u dan vertices v yang belum dikunjungi.
    4. Tetapkan v sebagai current vertices u. Tandai v telah dikunjungi.
    5. Jika semua vertices sudah dikunjungi, maka hentikan. Jika tidak, lanjutkan ke langkah 3.
```